

# DOCUMENTO TÉCNICO DEL TFM

## Predicción de churn en e-commerce

Código, arquitectura ML, pruebas, MLOps, API y preparación para producto

**Propósito:** convertir el diseño metodológico del TFM en una guía de implementación reproducible, defendible académicamente y preparada para una futura explotación como servicio.

**Dataset candidato:** UCI Online Retail II. El dataset debe auditarse con el archivo original antes de fijar cifras definitivas, reglas de limpieza, horizonte de churn y resultados.

**Nota:** este documento define la arquitectura y el código base. No sustituye la auditoría empírica del dataset real ni permite inventar resultados que todavía no se hayan ejecutado.

# Índice técnico

| Sección | Contenido                                  |
|---------|--------------------------------------------|
| 1       | Objetivo técnico y principios de diseño    |
| 2       | Arquitectura completa del proyecto         |
| 3       | Estructura del repositorio                 |
| 4       | Configuración y reproducibilidad           |
| 5       | Carga, limpieza y validación de datos      |
| 6       | Snapshots temporales y definición de churn |
| 7       | Feature engineering: RFM y comportamiento  |
| 8       | Prevención de leakage                      |
| 9       | Split temporal y validación                |
| 10      | Pipeline de machine learning               |
| 11      | Modelos y tratamiento del desbalance       |
| 12      | Métricas, calibración y threshold          |
| 13      | SHAP y explicabilidad                      |
| 14      | MLflow y trazabilidad de experimentos      |
| 15      | Pruebas automatizadas con pytest           |
| 16      | API con FastAPI                            |
| 17      | Docker y despliegue                        |
| 18      | Logging, errores y seguridad               |
| 19      | Fases de implementación                    |
| 20      | Definition of Done                         |
| 21      | Referencias técnicas                       |

# 1. Objetivo técnico y principios de diseño

El objetivo del código es construir un sistema que, dado un cliente y una fecha de corte T, genere variables únicamente con información disponible hasta T y estime la probabilidad de que el cliente entre en churn durante un horizonte futuro H. La unidad de observación será **cliente + snapshot temporal**, no la transacción individual.

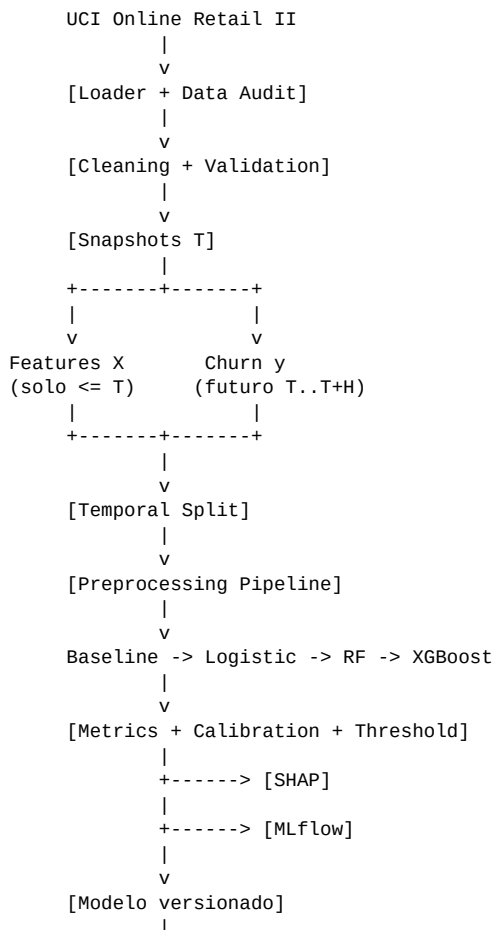
## Principios que deben mantenerse durante todo el proyecto:

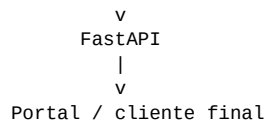
- **Temporalidad:** ninguna variable de entrada puede utilizar información posterior a T.
- **Reproducibilidad:** misma versión de datos + misma configuración + misma semilla deben producir resultados equivalentes.
- **Separación:** train, validation y test deben respetar el orden temporal.
- **Pipeline único:** el preprocesamiento debe viajar junto con el modelo para evitar diferencias entre entrenamiento e inferencia.
- **Probabilidad útil:** no basta con clasificar 0/1; el sistema debe producir una probabilidad calibrada y permitir elegir un threshold de negocio.
- **Auditabilidad:** cada experimento debe poder reconstruirse mediante parámetros, métricas, artefactos y versión del código.
- **Testabilidad:** las reglas críticas, especialmente churn, split y leakage, deben tener pruebas automatizadas.

La primera versión no debe intentar construir todo el producto. El orden correcto es: **datos** → **features** → **etiqueta** → **evaluación temporal** → **modelos** → **explicabilidad** → **tracking** → **API** → **producto web**.

## 2. Arquitectura completa del proyecto

### Arquitectura conceptual:





La idea clave es que el modelo que se sirve en producción sea exactamente el objeto entrenado y evaluado, incluyendo el preprocesamiento. Así se reduce el riesgo de que el API transforme las variables de una manera diferente a la utilizada durante el entrenamiento.

## Arquitectura de herramientas:

| Herramienta  | Función                      | Por qué                                      |
|--------------|------------------------------|----------------------------------------------|
| Python       | Lenguaje principal           | Ecosistema científico y ML maduro            |
| pandas       | Manipulación de tablas       | Adecuado para el dataset transaccional       |
| scikit-learn | Pipelines, modelos, métricas | Reproducibilidad y validación integrada      |
| XGBoost      | Modelo de boosting           | Fuerte baseline para datos tabulares         |
| SHAP         | Explicabilidad               | Explicar predicciones globales y por cliente |
| pytest       | Tests                        | Evitar regresiones y detectar leakage        |
| MLflow       | Experimentos/modelos         | Trazabilidad y versionado                    |
| FastAPI      | Servicio de predicción       | API validada y documentación OpenAPI         |
| Docker       | Empaquetado                  | Mismo entorno entre desarrollo y despliegue  |

### 3. Estructura del repositorio

```
tfm-churn-ecommerce/
├── data/
│   ├── raw/
│   ├── interim/
│   └── processed/
├── notebooks/
│   ├── 01_audit.ipynb
│   ├── 02_churn_horizon.ipynb
│   └── 03_model_analysis.ipynb
├── src/
│   ├── config/
│   │   └── settings.py
│   ├── data/
│   │   ├── loader.py
│   │   ├── cleaning.py
│   │   └── validation.py
│   ├── labels/
│   │   └── churn.py
│   ├── features/
│   │   ├── rfm.py
│   │   ├── behavioral.py
│   │   └── pipeline.py
│   ├── splitting/
│   │   └── temporal_split.py
│   ├── models/
│   │   ├── baseline.py
│   │   ├── logistic.py
│   │   ├── random_forest.py
│   │   └── xgboost_model.py
│   ├── evaluation/
│   │   ├── metrics.py
│   │   ├── calibration.py
│   │   └── threshold.py
│   ├── explainability/
│   │   └── shap_explainer.py
│   └── api/
│       ├── main.py
│       ├── schemas.py
│       └── predictor.py
├── tests/
│   ├── test_cleaning.py
│   ├── test_features.py
│   ├── test_churn.py
│   ├── test_temporal_split.py
│   ├── test_no_leakage.py
│   ├── test_model.py
│   └── test_api.py
├── scripts/
│   ├── 01_profile_data.py
│   ├── 02_build_dataset.py
│   ├── 03_train.py
│   └── 04_evaluate.py
├── Dockerfile
├── pyproject.toml
├── README.md
└── .gitignore
```

Separar **src/** de **notebooks/** es importante. Los notebooks sirven para exploración y gráficos; la lógica que debe ser reproducible debe vivir en módulos Python y poder ejecutarse desde scripts o tests.

Regla práctica: si una transformación es necesaria para producir el dataset final o entrenar el modelo, debe existir como función/módulo testeable, aunque además se documente visualmente en un notebook.

### 4. Configuración y reproducibilidad

La configuración debe centralizar las decisiones que afectan al experimento: horizonte H, frecuencia de snapshots, semilla, columnas, rutas y parámetros principales.

```
# src/config/settings.py
from dataclasses import dataclass
from pathlib import Path

@dataclass(frozen=True)
```

```
class Settings:
    random_seed: int = 42
    churn_horizon_days: int = 90
    snapshot_frequency: str = "MS"
    data_path: Path = Path("data/raw/online_retail_II.xlsx")
    target_col: str = "churn"

SETTINGS = Settings()
```

## En el TFM deben registrarse además:

- Versión/hash del archivo de datos utilizado.
- Versión de Python y dependencias instaladas.
- Semilla aleatoria.
- Definición exacta de snapshot, H y churn.
- Reglas de cancelaciones/devoluciones.
- Lista final de features.
- Algoritmo, hiperparámetros y threshold.
- Commit de Git asociado al experimento.

## Ejemplo de dependencia reproducible:

```
# pyproject.toml (estructura conceptual)
[project]
name = "tfm-churn-ecommerce"
version = "0.1.0"
dependencies = [
    "pandas",
    "numpy",
    "scikit-learn",
    "xgboost",
    "shap",
    "mlflow",
    "fastapi",
    "pydantic",
    "uvicorn",
    "pytest"
]
```

No conviene fijar versiones arbitrarias en el documento antes de crear el entorno real. Una vez creado, se deben congelar las versiones efectivamente utilizadas y anexarlas al TFM.

## 5. Carga, limpieza y validación de datos

La primera ejecución debe ser un **Data Audit**, no un entrenamiento. El archivo original de UCI debe conservarse en `data/raw` y nunca modificarse manualmente.

```
# src/data/loader.py
import pandas as pd

def load_online_retail(path: str) -> pd.DataFrame:
    xls = pd.ExcelFile(path)
    frames = [pd.read_excel(path, sheet_name=s) for s in xls.sheet_names]
    df = pd.concat(frames, ignore_index=True)
    return df
```

### Validaciones mínimas:

- Columnas esperadas y tipos.
- InvoiceDate convertible a datetime.
- CustomerID con el nivel de missing real.
- InvoiceNo de cancelación identificado según la regla del dataset.
- Quantity y UnitPrice revisados para ceros/negativos.

- Duplicados exactos.
- Rango temporal.
- Clientes y facturas únicas.
- Distribución de transacciones por cliente.
- Países y productos.
- Comportamiento de compras y huecos entre compras.

## Ejemplo de validación estructural:

```
REQUIRED = {
    "InvoiceNo", "StockCode", "Description",
    "Quantity", "InvoiceDate", "UnitPrice",
    "CustomerID", "Country"
}

def validate_schema(df):
    missing = REQUIRED - set(df.columns)
    if missing:
        raise ValueError(f"Missing columns: {missing}")

    if not pd.api.types.is_datetime64_any_dtype(df["InvoiceDate"]):
        raise TypeError("InvoiceDate must be datetime")
```

La limpieza no debe decidir de forma silenciosa qué hacer con devoluciones. Debe producir una decisión documentada y medible. Para este proyecto se recomienda calcular inicialmente **Monetary bruto** y **Monetary neto**, comparar ambos y congelar la regla final después de la auditoría.

## 6. Snapshots temporales y definición de churn

Esta es una de las partes metodológicamente más importantes. Para cada fecha de corte T se construye una fotografía del cliente utilizando solo el pasado. Después se observa una ventana futura para determinar y = churn.

```
def build_snapshot_dates(min_date, max_date, freq="MS"):
    return pd.date_range(
        start=min_date.normalize(),
        end=max_date.normalize(),
        freq=freq
    )
```

### Definición conceptual:

```
X(customer, T) = features(transactions where InvoiceDate <= T)

y(customer, T, H) =
    1 si NO existe compra válida en (T, T + H]
    0 si existe al menos una compra válida en (T, T + H]
```

La palabra **válida** debe quedar definida en el código. Por ejemplo, una cancelación no debería convertirse automáticamente en una nueva compra solo porque existe un InvoiceNo.

La selección de H no debe ser arbitraria. Se propone comparar 60, 90 y 120 días y estudiar empíricamente los intervalos entre compras; si resulta adecuado, complementar con análisis de supervivencia/Kaplan-Meier.

### Función de ejemplo:

```
def make_churn_label(
    customer_history,
    snapshot_date,
    horizon_days,
):
    future_end = snapshot_date + pd.Timedelta(days=horizon_days)

    future = customer_history[
        (customer_history["InvoiceDate"] > snapshot_date) &
        (customer_history["InvoiceDate"] <= future_end)
    ]

    return int(future.empty)
```

El dataset final debe tener una fila por **customer\_id + snapshot\_date**. Esto permite estudiar la evolución del riesgo a lo largo del tiempo.

## 7. Feature engineering: RFM y comportamiento

RFM es el núcleo inicial: Recency, Frequency y Monetary. Se amplía con variables conductuales que aporten señal temporal.

```
def build_rfm(transactions, snapshot_date):
    hist = transactions[
        transactions["InvoiceDate"] <= snapshot_date
    ].copy()

    rfm = (
        hist.groupby("CustomerID")
        .agg(
            last_purchase=("InvoiceDate", "max"),
            frequency=("InvoiceNo", "nunique"),
            monetary=("NetAmount", "sum"),
            unique_products=("StockCode", "nunique"),
        )
        .reset_index()
    )

    rfm["recency"] = (
        snapshot_date - rfm["last_purchase"]
    ).dt.days

    return rfm
```

### Features adicionales candidatas:

- Número de órdenes.
- Ticket promedio.
- Unidades compradas.
- Productos únicos.
- Días desde la primera compra.
- Intervalo medio entre compras.
- Intervalo mediano entre compras.
- Desviación de los intervalos.
- Gasto reciente frente a gasto histórico.
- Frecuencia reciente frente a frecuencia histórica.
- Tendencia de gasto.
- Número de compras en ventanas recientes, por ejemplo 30/60/90 días, si el diseño final lo justifica.

Importante: cada ventana debe terminar en T. Si se calcula una feature de los últimos 90 días, la ventana es [T-90, T], nunca [T-90, T+90].

### Función ejemplo para una ventana:

```
def count_orders_last_days(df, snapshot_date, days):
    start = snapshot_date - pd.Timedelta(days=days)
    mask = (
        (df["InvoiceDate"] > start) &
        (df["InvoiceDate"] <= snapshot_date)
    )
    return df.loc[mask].groupby("CustomerID")["InvoiceNo"].nunique()
```

## 8. Prevención de leakage

El leakage es el riesgo técnico más grave del proyecto. Un modelo puede obtener métricas excelentes y ser metodológicamente inválido si las features contienen información del futuro.

Regla formal: para cada fila (cliente, T), cualquier transacción utilizada en X debe satisfacer **InvoiceDate**  $\leq$  T. La etiqueta sí puede usar el futuro, porque precisamente define el evento que queremos predecir.

### Test automático mínimo:

```
def assert_features_do_not_use_future(
    feature_transactions,
    snapshot_date
):
    future = feature_transactions[
        feature_transactions["InvoiceDate"] > snapshot_date
    ]
    assert future.empty, (
        "Leakage detected: future transactions "
        "entered the feature calculation."
    )
```

Test más fuerte: construir un dataset sintético con una compra deliberadamente colocada después de T y comprobar que modificar esa compra no cambia ninguna feature de X. En cambio, sí debe poder cambiar la etiqueta y.

```
def test_future_transaction_cannot_change_features():
    base = make_synthetic_transactions()
    x1 = build_features(base, snapshot_date=T)

    modified = base.copy()
    modified.loc[len(modified)] = future_transaction

    x2 = build_features(modified, snapshot_date=T)

    assert_frame_equal(
        x1.sort_index(axis=1),
        x2.sort_index(axis=1)
    )
```

Este test es especialmente valioso para el TFM porque transforma una afirmación metodológica (“no hay leakage”) en una propiedad comprobable automáticamente.

## 9. Split temporal y validación

No se debe utilizar un `train_test_split` aleatorio como evaluación principal de este problema. La predicción debe respetar el orden temporal.

```
def temporal_split(df, date_col, train_end, valid_end):
    train = df[df[date_col] <= train_end].copy()

    valid = df[
        (df[date_col] > train_end) &
        (df[date_col] <= valid_end)
    ].copy()

    test = df[df[date_col] > valid_end].copy()

    return train, valid, test
```

Hay además una cuestión específica: un mismo cliente puede aparecer en muchos snapshots. Esto no es automáticamente leakage si el futuro está correctamente separado, pero sí crea dependencia entre observaciones. Por eso el informe debe explicar la estrategia temporal y analizar la presencia repetida de clientes.

### Una evaluación robusta puede incluir:

- Train: snapshots iniciales.
- Validation: periodo inmediatamente posterior.
- Test: periodo final completamente no utilizado para seleccionar modelo/threshold.
- Análisis de sensibilidad con distintas ventanas temporales.

- Control de que ninguna decisión del test se use para volver a entrenar o elegir hiperparámetros.

## 10. Pipeline de machine learning

El preprocesamiento debe estar dentro de un Pipeline de scikit-learn. Esto evita olvidar transformaciones durante validación o inferencia.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

numeric_features = [
    "recency", "frequency", "monetary",
    "avg_order_value", "unique_products"
]

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipe, numeric_features),
])

model = Pipeline([
    ("preprocessor", preprocessor),
    ("model", LogisticRegression(
        max_iter=2000,
        class_weight="balanced",
        random_state=42
    ))
])
```

Ventaja académica: el Pipeline hace explícita la secuencia de transformaciones y permite que la validación aplique las transformaciones aprendidas solo sobre los datos correspondientes al entrenamiento.

El mismo Pipeline debe ser el objeto que se guarda y posteriormente se usa en FastAPI.

## 11. Modelos y tratamiento del desbalance

La comparación debe comenzar por un baseline sencillo. Esto permite saber si un modelo complejo realmente aporta valor.

### Secuencia recomendada:

- DummyClassifier: referencia mínima.
- Logistic Regression: baseline interpretable.
- Random Forest: no lineal y robusto para tabular.
- XGBoost/Gradient Boosting: candidato principal para datos tabulares.
- Deep learning: opcional, solo si existe justificación y mejora demostrable.

El desbalance debe medirse antes de decidir la técnica. No hay que aplicar SMOTE por defecto.

### Experimentos comparables:

- Sin tratamiento.
- class\_weight='balanced' cuando el modelo lo soporte.
- Undersampling dentro del entrenamiento, si se justifica.
- SMOTE únicamente sobre el conjunto de entrenamiento o dentro de una estrategia de CV que evite contaminar validación/test.

- Threshold tuning como mecanismo independiente del balanceo.

La métrica principal recomendada para comparar modelos en presencia de desbalance es PR-AUC, complementada por ROC-AUC, recall, precision y F1. La elección final debe relacionarse con el coste de falsos positivos y falsos negativos.

## 12. Métricas, calibración y threshold

```
from sklearn.metrics import (
    average_precision_score,
    roc_auc_score,
    precision_score,
    recall_score,
    f1_score
)

def evaluate_binary(y_true, proba, threshold=0.5):
    pred = (proba >= threshold).astype(int)

    return {
        "pr_auc": average_precision_score(y_true, proba),
        "roc_auc": roc_auc_score(y_true, proba),
        "precision": precision_score(y_true, pred),
        "recall": recall_score(y_true, pred),
        "f1": f1_score(y_true, pred),
    }
```

No debe asumirse que 0.5 es el threshold óptimo. Si el objetivo es contactar a un número limitado de clientes, puede ser más útil seleccionar el top-k% por probabilidad o un threshold que cumpla una restricción de recall/precision/coste.

### Calibración:

Una probabilidad de 0.80 debería significar aproximadamente que, entre clientes con predicción 0.80, cerca del 80% presentan el evento según la definición y horizonte estudiados. Por eso conviene evaluar calibración y, si es necesario, utilizar herramientas como CalibratedClassifierCV sobre un conjunto apropiado.

```
from sklearn.calibration import CalibratedClassifierCV

calibrated = CalibratedClassifierCV(
    estimator=base_model,
    method="sigmoid",
    cv=5
)
```

En el informe final se deben presentar tanto métricas discriminativas como calibración y una matriz de confusión asociada al threshold elegido.

## 13. SHAP y explicabilidad

SHAP se utilizará después de seleccionar el modelo candidato. Su objetivo no es adornar el TFM, sino responder dos preguntas: **qué variables influyen globalmente** y **por qué un cliente concreto recibe un riesgo alto/bajo**.

```
import shap

# Para modelos compatibles con TreeExplainer:
explainer = shap.TreeExplainer(trained_model)
shap_values = explainer(X_test)

# Después:
# shap.summary_plot(...)
# shap.waterfall_plot(...)
```

### Ejemplo de explicación que podría alimentar la API:

```
{
    "churn_probability": 0.82,
    "risk_level": "HIGH",
    "top_factors": [
        "high_recency",
```

```

        "declining_frequency",
        "low_recent_spend"
    ]
}

```

No se deben inventar explicaciones semánticas. Si SHAP indica que recency aumenta el riesgo, la aplicación puede traducirlo a una frase comprensible, pero la frase debe corresponder a la feature real y a su dirección.

## 14. MLflow y trazabilidad de experimentos

MLflow puede registrar parámetros, métricas, artefactos y modelos. Para el TFM esto permite demostrar que las comparaciones entre experimentos son trazables.

```

import mlflow

with mlflow.start_run():
    mlflow.log_param("churn_horizon_days", 90)
    mlflow.log_param("snapshot_frequency", "MS")
    mlflow.log_param("model", "xgboost")
    mlflow.log_param("random_seed", 42)

    mlflow.log_metric("pr_auc", pr_auc)
    mlflow.log_metric("roc_auc", roc_auc)
    mlflow.log_metric("f1", f1)

    # mlflow.log_artifact("reports/confusion_matrix.png")
    # mlflow.sklearn.log_model(model, "model")

```

### Run conceptual:

```

Experiment
  Run
    dataset_hash
    snapshot_definition
    churn_horizon
    feature_set
    model
    hyperparameters
    PR-AUC
    ROC-AUC
    calibration
    threshold
    plots
    model artifact

```

Recomendación para el TFM: utilizar MLflow localmente al principio. No hace falta montar una infraestructura compleja. Lo importante es que los experimentos queden registrados de forma consistente.

## 15. Pruebas automatizadas con pytest

Las pruebas deben cubrir las reglas que, si fallan, invalidarían el estudio.

```

tests/
  test_cleaning.py
  test_features.py
  test_churn.py
  test_temporal_split.py
  test_no_leakage.py
  test_model.py
  test_api.py

```

### Ejemplos:

```

def test_churn_is_one_without_future_purchase():
    df = make_customer_history()
    y = make_churn_label(
        df,
        snapshot_date=T,
        horizon_days=90
    )
    assert y == 1

def test_future_purchase_makes_churn_zero():

```

```

df = make_customer_history_with_future_purchase()
y = make_churn_label(
    df,
    snapshot_date=T,
    horizon_days=90
)
assert y == 0

def test_split_is_temporal():
    train, valid, test = temporal_split(
        dataset, "snapshot_date",
        train_end=TRAIN_END,
        valid_end=VALID_END
    )
    assert train["snapshot_date"].max() <= TRAIN_END
    assert valid["snapshot_date"].min() > TRAIN_END
    assert test["snapshot_date"].min() > VALID_END

```

Debe existir al menos un test dedicado exclusivamente a leakage. Este punto puede presentarse en el TFM como parte del mecanismo de garantía de calidad del pipeline.

## 16. API con FastAPI

Cuando el modelo esté validado, FastAPI permite convertirlo en un servicio HTTP. La API debe ser una capa fina: validar entrada, ejecutar el pipeline, devolver probabilidad, nivel de riesgo y versión del modelo.

```

# src/api/schemas.py
from pydantic import BaseModel, Field

class CustomerFeatures(BaseModel):
    recency: float = Field(ge=0)
    frequency: float = Field(ge=0)
    monetary: float
    avg_order_value: float
    unique_products: float = Field(ge=0)

class PredictionResponse(BaseModel):
    churn_probability: float
    risk_level: str
    model_version: str

# src/api/main.py
from fastapi import FastAPI
from .schemas import CustomerFeatures, PredictionResponse

app = FastAPI(
    title="E-commerce Churn Prediction API",
    version="1.0.0"
)

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict", response_model=PredictionResponse)
def predict(payload: CustomerFeatures):
    probability = predictor.predict(payload)
    risk = risk_from_probability(probability)

    return PredictionResponse(
        churn_probability=probability,
        risk_level=risk,
        model_version="1.0.0"
    )

```

FastAPI genera documentación OpenAPI y una interfaz interactiva para probar los endpoints. Esto es útil tanto técnicamente como para la demostración del TFM.

## 17. Docker y despliegue

Docker se incorpora después de estabilizar el código. Su finalidad es empaquetar el servicio con un entorno reproducible.

```
# Dockerfile
FROM python:3.12-slim

WORKDIR /app

COPY pyproject.toml .
RUN pip install --no-cache-dir .

COPY src ./src
COPY models ./models

EXPOSE 8000

CMD ["uvicorn", "src.api.main:app",
    "--host", "0.0.0.0",
    "--port", "8000"]
```

## Flujo de despliegue:

```
Código + dependencias
|
v
Docker image
|
v
FastAPI service
|
v
POST /predict
|
v
probability + risk
|
v
portal / cliente
```

Para el TFM no es necesario contratar infraestructura cloud desde el inicio. Una demo local con Docker es suficiente para demostrar productización; un despliegue público puede plantearse como extensión.

## 18. Logging, errores y seguridad

La API no debe devolver trazas internas ni fallar silenciosamente. Debe distinguir errores de validación, modelo no disponible y errores inesperados.

```
import logging

logger = logging.getLogger("churn_api")

@app.get("/health")
def health():
    return {"status": "ok"}

# Conceptualmente:
try:
    probability = predictor.predict(payload)
except ValueError as exc:
    logger.warning("Invalid prediction input: %s", exc)
    raise HTTPException(status_code=400, detail="Invalid input")
except Exception:
    logger.exception("Prediction failed")
    raise HTTPException(
        status_code=500,
        detail="Prediction service unavailable"
    )
```

### Buenas prácticas mínimas:

- No almacenar datos personales innecesarios.
- No registrar CustomerID u otros identificadores en logs si no son necesarios.
- Validar rangos y tipos con Pydantic.
- No incluir secretos en Git.

- Usar variables de entorno para configuración sensible.
- Cargar únicamente modelos/artifacts de confianza.
- Separar datos de desarrollo de datos reales.
- Si el servicio se expone públicamente, añadir autenticación, HTTPS y límites de uso.

## 19. Fases de implementación

Para evitar dispersión tecnológica, se propone este orden.

| Fase | Entregable                       | Tecnología             |
|------|----------------------------------|------------------------|
| 1    | Auditoría reproducible del UCI   | Python + pandas        |
| 2    | Reglas de limpieza/cancelaciones | pandas + pytest        |
| 3    | Snapshots + churn 60/90/120      | pandas + tests         |
| 4    | RFM + features conductuales      | pandas                 |
| 5    | Split temporal + leakage tests   | scikit-learn + pytest  |
| 6    | Baselines y modelos              | scikit-learn + XGBoost |
| 7    | Métricas/calibración/threshold   | scikit-learn           |
| 8    | SHAP                             | SHAP                   |
| 9    | Tracking                         | MLflow                 |
| 10   | Modelo final reproducible        | MLflow/artifact        |
| 11   | API                              | FastAPI + Pydantic     |
| 12   | Empaquetado                      | Docker                 |
| 13   | Portal                           | Frontend + API         |

La fase 13 es opcional para la entrega académica inicial. El portal debe construirse cuando el modelo ya tenga una definición estable y una evaluación satisfactoria.

## 20. Definition of Done

El TFM técnico puede considerarse listo cuando se cumplen estas condiciones:

- El dataset utilizado está identificado y versionado/hash registrado.
- Existe un script reproducible de auditoría.
- Las reglas de cancelaciones/devoluciones están documentadas y testeadas.
- El horizonte H está justificado empíricamente.
- Existen múltiples snapshots.
- Features y etiqueta están separadas temporalmente.
- Existe un test automatizado de leakage.
- Train/validation/test respetan el tiempo.
- Hay baseline y al menos dos modelos competitivos.
- Se reportan PR-AUC, ROC-AUC, precision, recall y F1.

- Se evalúa calibración.
- El threshold final está justificado.
- Se incluye análisis SHAP del modelo final.
- MLflow o mecanismo equivalente registra experimentos.
- El modelo final incluye el preprocesamiento.
- Existe suite pytest para las reglas críticas.
- FastAPI expone /health y /predict.
- La API valida inputs y gestiona errores.
- El entorno puede reproducirse mediante dependencias fijadas.
- Docker permite ejecutar el servicio de forma reproducible.

Si el portal web se incorpora, debe considerarse una capa de consumo del modelo, no una sustitución de la validación científica.

## 21. Referencias técnicas

### Dataset

UCI Machine Learning Repository — Online Retail II, Dataset ID 502. Fuente oficial utilizada como referencia para el dataset candidato y su descripción.

### Machine Learning

scikit-learn Documentation — Pipeline, model evaluation, preprocessing y probability calibration.

XGBoost Documentation — Gradient boosting para clasificación tabular.

SHAP Documentation — Explainable AI y valores SHAP.

### MLOps

MLflow Documentation — Tracking, model logging, artifacts y model management.

### API

FastAPI Documentation — OpenAPI, Pydantic, validation y documentación interactiva.

### Testing

pytest Documentation — fixtures, assertions y estructura de tests.

### Literatura relacionada

Literatura reciente sobre churn en e-commerce y RFM, incluyendo trabajos de 2023–2026 sobre RFM temporal, modelos de deep learning, ensembles, SMOTE, SHAP y MLOps. La bibliografía definitiva del TFM debe incluir las referencias académicas completas y DOI cuando corresponda.

Documento preparado como guía técnica de implementación. Los resultados numéricos del TFM deben generarse únicamente después de ejecutar la auditoría y el pipeline sobre el archivo real de UCI.